

EXAM QUESTION PATTERN ANALYZER

PROJECT REPORT

A Flask-based application for extracting, processing and analyzing question papers using NLP, OCR, TF-IDF similarity and topic-frequency analysis.

Project	Exam Question Pattern Analyzer
Student	Soumyadip Porel
Course	BCA
Backend	Python + Flask
NLP / ML	NLTK, scikit-learn, TF-IDF, Cosine Similarity
Document Processing	PyMuPDF + Tesseract OCR
Frontend	HTML, CSS, JavaScript + Chart.js
API Dependency	No OpenAI API required
Repository	github.com/Soumyadip-gif/Exam_Question_Pattern_Analyzer

Prepared for academic project documentation and presentation.

1. Abstract

The Exam Question Pattern Analyzer is a practical web application designed to analyze question papers and identify useful examination patterns. The system accepts normal PDFs, scanned PDFs and common image formats. It extracts text using PyMuPDF when selectable text is available and automatically falls back to OCR for scanned documents. Extracted questions are cleaned and tokenized with NLTK, assigned to predefined academic topics, and compared using TF-IDF and cosine similarity. The application also calculates topic frequency, highlights frequently occurring topics, identifies highly similar questions, and presents results through a browser-based dashboard with a Chart.js topic distribution chart. The current implementation is built with Python and Flask and does not require an OpenAI API.

2. Introduction

Students often study previous question papers to understand which subjects and concepts appear repeatedly. Doing this manually across several papers can be time-consuming and can make recurring topics or similar questions difficult to identify. This project automates a significant part of that process by converting question-paper content into structured information and displaying analysis results in one interface. It demonstrates practical use of Flask web development, document processing, OCR, natural language processing, information retrieval and similarity analysis.

3. Problem Statement

Manual examination of previous question papers requires significant time and effort. Repeated questions and recurring topics may be difficult to notice when papers are analyzed separately. Scanned question papers often contain images rather than selectable text, making ordinary PDF extraction insufficient. A student-friendly system is therefore useful for combining extraction, topic detection, similarity analysis and visual reporting.

4. Objectives

The main objectives are to accept PDF and image question papers; extract text from normal and scanned documents; detect numbered questions; clean question text and extract keywords; classify questions into predefined academic topics; measure similarity using TF-IDF and cosine similarity; calculate topic frequency; identify frequently occurring topics; and present results through a clear web dashboard and chart.

5. Technology Stack

Layer	Technology	Purpose
Backend	Python + Flask	Web server, routing and analysis workflow
PDF	PyMuPDF	Selectable PDF text extraction and page rendering
OCR	Tesseract + pytesseract	Text extraction from scanned pages/images
Images	Pillow	Image conversion and OCR preprocessing
NLP	NLTK	Tokenization and stop-word removal
Similarity	scikit-learn	TF-IDF and cosine similarity
Frontend	HTML + CSS + JavaScript	Interface and result presentation
Charts	Chart.js	Topic distribution visualization
Version Control	Git + GitHub	Source-code versioning and hosting

6. System Architecture

The application follows a modular processing pipeline. A user uploads a question paper through the Flask interface. The application determines the file type, extracts text directly from a normal PDF or invokes OCR when necessary. The extracted text is passed through question extraction, NLP preprocessing and topic detection. The resulting question objects are then used for similarity and pattern analysis. Flask finally sends the structured results to the results page.

- Input: PDF / JPG / JPEG / PNG
- Text extraction: PyMuPDF
- OCR fallback: Tesseract
- Question extraction: regular-expression based parser
- NLP processing: cleaning, tokenization and stop-word removal
- Topic detection: predefined academic keyword matching
- Similarity: TF-IDF + cosine similarity
- Pattern analysis: topic frequency and insights
- Presentation: Flask templates + CSS + JavaScript + Chart.js

7. Project Structure

```
Exam-Question-Pattern-Analyzer/  
■■■ app.py  
■■■ requirements.txt  
■■■ test_nlp.py  
■■■ test_ocr.py  
■■■ test_pattern.py  
■■■ test_pdf_ocr.py  
■■■ test_similarity.py  
■■■ test_topic.py  
■■■ templates/  
■ ■■■ index.html  
■ ■■■ results.html  
■■■ static/
```

- ■■■ css/style.css
- ■■■ js/script.js
- processing/
- pdf_extractor.py
- image_ocr.py
- question_extractor.py
- nlp_processor.py
- topic_detector.py
- similarity_analyzer.py
- pattern_analyzer.py

8. Module Description

app.py

Main Flask application. Handles uploads, chooses extraction methods, coordinates processing modules and renders results.

pdf_extractor.py

Extracts selectable PDF text with PyMuPDF and provides PDF-page OCR processing when direct extraction is insufficient.

image_ocr.py

Preprocesses images and sends them to Tesseract OCR.

question_extractor.py

Uses regular expressions to split extracted text into numbered questions.

nlp_processor.py

Cleans question text, tokenizes it, removes English stop words and returns keywords.

topic_detector.py

Uses predefined keyword groups to assign academic topics.

similarity_analyzer.py

Creates TF-IDF vectors and calculates cosine similarity between questions.

pattern_analyzer.py

Calculates topic frequency and supports important-topic and pattern-insight generation.

results.html

Displays analyzed questions, topics, similarity results, insights and visualizations.

script.js

Creates topic-frequency bars and renders the Chart.js topic distribution chart.

9. Detailed Workflow

- Upload — The user selects a question paper from the web interface.
- File validation — Flask checks the file extension against supported PDF and image formats.
- Text extraction — PyMuPDF attempts direct extraction for PDFs; images use OCR.
- OCR fallback — If a PDF contains little or no selectable text, its pages are rendered and passed through Tesseract.
- Question extraction — Numbered question patterns are detected and separated.
- NLP processing — Questions are cleaned, tokenized and converted into keyword lists.
- Topic detection — Predefined topic keywords are matched against each question.
- Similarity analysis — TF-IDF vectors and cosine similarity identify highly similar question pairs.
- Pattern analysis — Topic counts and summary insights are generated.

- Visualization — Results are displayed through cards, topic bars and a Chart.js chart.

10. NLP Processing

The NLP module converts text to lowercase, removes non-alphabetic characters, tokenizes the remaining text with NLTK, removes common English stop words and excludes very short tokens. The output retains the original question, cleaned text and keywords. This representation is used by the topic and similarity stages.

11. Topic Detection

The current topic detector is rule-based rather than a trained machine-learning classifier. It maintains keyword groups for DBMS, Operating System, Java, Software Engineering, Computer Network and Data Structure. Matching keywords increase a topic score; the highest-scoring topic is selected. If no topic keyword is matched, the question is assigned to General / Unknown.

12. Similarity Analysis

The similarity module uses TF-IDF (Term Frequency-Inverse Document Frequency) to represent questions numerically. Cosine similarity is calculated between question vectors. The application currently uses a threshold of 0.50 to report question pairs as similar or repeated, and displays the resulting score.

13. OCR and Scanned Document Support

Direct PDF text extraction cannot reliably read pages that contain only images. The application therefore checks the amount of extracted text and activates an OCR path when it is below the configured threshold. PDF pages are rendered at increased resolution and processed by the OCR module. Image uploads can also use the same OCR pipeline. Preprocessing includes RGB conversion, 2× resizing, grayscale conversion, contrast enhancement, sharpening and thresholding.

14. Pattern Analysis and Dashboard

- Total Questions — number of extracted question records.
- Most Asked Topic — topic with the highest detected frequency.
- Questions in Top Topic — count associated with the most frequent topic.
- Topics Detected — number of topic categories appearing in the analysis.
- Repeated Questions — count based on similarity results.
- Topic Frequency Analysis — visual bars showing questions per topic.
- Most Important Topics — frequently occurring topics shown as cards.
- Topic Distribution Chart — Chart.js bar chart generated from Flask topic-frequency data.

15. User Interface

The application provides an upload page followed by a results dashboard. The results page presents each extracted question, cleaned text, keywords and detected topic. Separate sections show similarity results, topic frequency, pattern insights, important topics and a graphical topic distribution. The interface uses HTML templates, CSS and JavaScript, with Chart.js for visualization.

16. Testing Performed

Test	Purpose
test_nlp.py	Verified text cleaning, tokenization, stop-word removal and keyword extraction.
test_topic.py	Verified topic detection with representative academic questions.
test_similarity.py	Verified TF-IDF and cosine similarity calculations.
test_pattern.py	Verified topic-frequency analysis.
test_pdf_ocr.py	Verified OCR processing for scanned PDF pages.
test_ocr.py	Used for OCR-related testing.
Flask upload workflow	Verified upload, extraction, analysis and result rendering.
Chart.js integration	Verified topic distribution chart rendering.

17. Installation and Execution

A typical local setup is:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Tesseract OCR must also be installed on the system for scanned PDFs and image OCR.

18. Limitations

- Question extraction depends on recognizable numbering patterns and may need improvement for heavily distorted OCR text.
- OCR can introduce spelling, numbering and formatting errors, especially in low-quality scans.
- The topic detector is keyword-based and limited to its predefined vocabulary.
- TF-IDF cosine similarity is based on lexical overlap and does not provide full semantic understanding.
- The current workflow analyzes an uploaded paper at a time; multi-paper historical trend analysis can be added later.
- The current application does not include user accounts or persistent analysis history.

19. Future Enhancements

- Improve OCR accuracy using stronger preprocessing and layout-aware extraction.
- Improve sub-question extraction for formats such as (i), (ii), (iii), etc.
- Add semantic embeddings for more meaning-aware similarity detection.
- Train a machine-learning topic classifier using a labeled academic question dataset.
- Support multiple question papers for semester/year-wise trend analysis.
- Add database storage for uploaded-paper metadata and historical results.
- Add authentication and role-based access for multi-user deployment.
- Add downloadable analysis reports and richer charts.
- Deploy the application to a cloud platform.

20. Conclusion

The Exam Question Pattern Analyzer demonstrates how multiple practical computing techniques can be combined into one academic application. Flask provides the web layer, PyMuPDF and Tesseract handle document extraction, NLTK performs basic NLP preprocessing, scikit-learn supports TF-IDF similarity analysis, and JavaScript/Chart.js provide visualization. The workflow reduces manual effort when inspecting question papers and provides a structured view of topics, keywords, similar questions and recurring patterns. The project also provides a foundation for future extensions involving machine learning, semantic analysis, multi-paper trend detection and persistent storage.

21. Project Repository

GitHub: github.com/Soumyadip-gif/Exam_Question_Pattern_Analyzer

END OF REPORT